

Phase 2 - explanation of logic and approach

Data Vortex Round 1 Phase 2 · how each of the 21 challenges was answered, and why that reading was chosen over the alternatives.

Team	Forge-X
Team head	Kunal Choudhary
Covers	E1-E5, M1-M5, H1-H6, each with its verification queries
Database	output/social_engine.db - posts, users, post_tags, raw_posts, post_anomalies + views
Reproduce	./run_all.sh

Where this explanation comes from

The reasoning below is the LOGIC note attached to each query inside queries/phase2_challenges.sql, rendered from the same file that was executed. It cannot drift from the code, because it is the code's own annotation.

Approach

- **One engagement definition.** All 21 answers use the same NULL-safe expression. SQL's bare + is NULL-propagating: `AVG(likes+shares+comments)` would have silently dropped 1,532 rows and changed the population and the metric at the same time.
- **Missing data is never filled.** 1,532 posts (15.0%) have no recorded like count and the source never explained why, so it stays missing. The queries filter where the challenge demands it and the comparison is undefined without it - M5 and H6 both rest on this.
- **A data gap is not a category.** The 1,541 posts whose platform label was lost are kept for volume questions and excluded from every per-platform rate, so an intake failure can never be published as a platform finding.
- **Deterministic ordering.** Post counts and engagement totals tie constantly at this size, so every LIMIT is paired with a tie-break (`post_id / user_id`). Without it the membership of a top-10 would depend on storage order.
- **Every answer is attacked before it is published.** The 17 companion queries apply the statistical test, the alternative reading, or the sensitivity sweep. Where a headline does not survive, the report says so - six findings were withdrawn this way.

EASY questions

E1 - Platform Popularity

Question. Determine which social media platform has the highest number of posts. Ignore posts where the platform is missing. Return the platform name and number of posts.

Logic. R3 applies. 1,846 intake rows arrived with the platform cell blank or holding the literal 'NULL'; after de-duplication 1,541 of them survive in the analysis table as 'Unspecified'. The challenge says to ignore them, and they are ignored BEFORE ranking -- not filtered after -- so the ordering cannot be topped by a gap. The ORDER BY carries the platform name as a second key so the single returned row is stable.

platform	n_posts
YouTube	1770

Returns: 1 row: platform=YouTube, n_posts=1770

E1b - Platform Popularity

Question. (context for E1) The spread across the five labelled platforms is small. Report the full league table and test the gap.

Logic. A ranking alone cannot distinguish "YouTube leads" from "five platforms tied and YouTube came out on top of the noise". The chi-square goodness-of-fit statistic compares the observed post counts against the uniform expectation (total/k) that a null model of "no platform preference" would produce; $df = 5 - 1 = 4$ and the 5% critical value is 9.488. SUM(...) OVER () totals the per-platform terms without a second pass. This is the same discipline Phase 1 applied to the platform spread, applied here inside SQL.

platform	n_posts	pct_share	expected_if_uniform	chi2_term	chi2_total	df	verdict_at_5pc t
YouTube	1770	20.39	1736.0	0.6659	2.6094	4	field is flat -- the leader is ahead only by noise
Facebook	1763	20.31	1736.0	0.4199	2.6094	4	field is flat -- the leader is ahead only by noise
Twitter	1742	20.07	1736.0	0.0207	2.6094	4	field is flat -- the leader is ahead only by noise
Reddit	1716	19.77	1736.0	0.2304	2.6094	4	field is flat -- the leader is ahead only by noise
Instagram	1689	19.46	1736.0	1.2725	2.6094	4	field is flat -- the leader is ahead only by noise

Returns: 5 rows, led by platform=YouTube, n_posts=1770

E2 - Most Engaged Posts

Question. Find the top 10 posts based on total engagement. Total engagement is defined as likes + shares + comments. Ignore posts where likes are missing.

Logic. "likes + shares + comments" is written with COALESCE on the two columns that are complete in this dataset (shares and comments have no NULLs at all -- only likes does), so the expression is defined by construction rather than by luck. The WHERE clause is the challenge's own instruction and is the same rule as R2: 1,532 posts have no like count, and adding 0 for them would rank a post on partial evidence. Ranking is by the computed total, so the tie-break on post_id is what makes position 10 stable across re-runs (R4).

post_id	user_id	platform	likes	shares	comments	total_engagement
ycjj5zzt7mvx	user_d9971ba6	Instagram	4983	1919	991	7893
wo7py9aljg3t	user_o8le7hqf	Reddit	4864	1981	948	7793
gmoeib832zbs	user_pe5yckyb	Facebook	4902	1880	982	7764
5kvuyvf38nqx	user_z0feut2e	YouTube	4923	1971	861	7755
pvgl3d8hj7jd	user_cs1uibwk	Instagram	4989	1840	909	7738

post_id	user_id	platform	likes	shares	comments	total_engagement
tdgjyylpua20	user_8nvzxsuj	Unspecified	4979	1932	812	7723

Returns: 10 rows, led by post_id=ycjj5ztt7mvx, user_id=user_d9971ba6

E2b - Most Engaged Posts

Question. (context for E2) A top-10 is only meaningful if its cut is meaningful. Quantify the cut.

Logic. The 10th-place score is read with OFFSET 9 rather than by hardcoding a number, so the metric survives any re-run that changes the data. If the gap between 1st and 10th is small, the "top 10" is a slice through a dense cloud and should be reported as such instead of as ten standout posts.

comparable_posts	best_engagement	tenth_place_engagement	posts_within_1pct_of_cut	best_over_tenth_pct
8689	7893	7583	14	4.09

Returns: 1 row: comparable_posts=8689, best_engagement=7893

E3 - Average Engagement By Platform

Question. Calculate the average likes, shares, and comments for each platform. Which platform generates the highest average total engagement?

Logic. AVG(likes) in SQLite skips NULLs, which is the correct behaviour here: 1,532 posts have no like count and averaging over the posts that DO have one is honest, whereas AVG(COALESCE(likes,0)) would silently average 1,532 fabricated zeros into every platform's like mean and bias the answer downward. `rows\_with\_likes` is carried alongside so the reader can see the denominator each average was computed over. The rank is computed over the same AVG(engagement) expression that is displayed, and 'Unspecified' is retained in the table for completeness but flagged so it cannot be mistaken for the winner (R3).

platform	n_posts	rows_with_likes	avg_likes	avg_shares	avg_comments	avg_total_engagement	engagement_rank	eligibility
Instagram	1689	1435	2504.8	1043.3	501.8	3673.2	1	labelled platform
Unspecified	1541	1323	2501.2	999.2	500.4	3646.9	2	DATA GAP -- not a platform, not eligible to win
Facebook	1763	1494	2527.3	986.8	507.6	3636.1	3	labelled platform
YouTube	1770	1495	2494.9	1009.0	505.3	3621.5	4	labelled platform
Reddit	1716	1473	2463.8	985.1	506.9	3606.9	5	labelled platform
Twitter	1742	1469	2432.4	1006.8	503.5	3561.5	6	labelled platform

Returns: 6 rows, led by platform=Instagram, n_posts=1689

E3b - Average Engagement By Platform

Question. (context for E3) Name the winning platform and justify it.

Logic. Two platforms can differ in the mean and still be indistinguishable, because each mean carries its own sampling error. The two-sample z-statistic divides the gap by $\sqrt{\text{var1}/n1 + \text{var2}/n2}$, with the variance written as $E[x^2] - E[x]^2$ (SQLite has no VAR function), and guards the square root with MAX(...,0) so floating-point dust cannot produce a negative variance. 'Unspecified' is excluded here because this query names a winner (R3). $|z| > 1.96$ is the 5% two-sided bar.

winner	winner_avg_engagement	runner_up	runner_up_avg_engagement	absolute_gap	gap_pct	z_stat	verdict
Instagram	3673.2	Facebook	3636.1	37.0	1.02	0.635	NOT significant at 5% -- the ranking is a coin toss

Returns: 1 row: winner=Instagram, winner_avg_engagement=3,673.2

E4 - Highly Shared But Poorly Liked

Question. Identify posts that received more than 1,500 shares but fewer than 500 likes. Return the post ID, platform, likes, shares, and comments.

Logic. Both bounds are strict ('more than', 'fewer than'), so the predicates are > and <, not >= and <= -- a boundary error here would be invisible in the output but wrong. A NULL like count fails `likes < 500` in SQL automatically (NULL comparisons are never true), which is the correct outcome: a post with no recorded likes cannot be shown to be "poorly liked". That is stated rather than relied on silently.

post_id	platform	likes	shares	comments
euvr0r10wrj6	Facebook	453	2000	408
oisz0jqm6qnn	Instagram	390	1999	858
f2e5kdfldedz	Unspecified	447	1997	641
mgv7p46wzpek	Reddit	252	1993	971
u1aa801qvxeu	Unspecified	162	1992	306
0nsga7zrxpvt	YouTube	390	1991	848

Returns: 214 rows, led by post_id=euvr0r10wrj6, platform=Facebook

E4b - Highly Shared But Poorly Liked

Question. (context for E4) 214 posts match. Decide whether the pattern is platform-specific or spread across the whole platform.

Logic. A raw count per platform would just re-report platform volume, so the query returns the RATE -- flagged posts as a percentage of that platform's own posts. Rates are what expose a platform-specific behaviour; counts only expose which platform posts more. 'Unspecified' is kept here because this is a volume/rate question and the posts are real (R3), but it is marked.

platform	n_posts	flagged_posts	pct_of_platform
Instagram	1689	44	2.61
Facebook	1763	42	2.38
Twitter	1742	39	2.24
Unspecified	1541	31	2.01
YouTube	1770	35	1.98
Reddit	1716	23	1.34

Returns: 6 rows, led by platform=Instagram, n_posts=1689

E5 - Users With Large Audiences

Question. Find all users with more than 40,000 followers. Display their user ID, location, language, and follower count.

Logic. 'more than 40,000' is strict, so the predicate is > 40000: a user sitting exactly on 40,000 is not in the answer. `location` is the source's own string, kept verbatim in the users table, so every row here can be matched character-for-character against the intake file. The tie-break on user_id makes the ordering total (R4).

user_id	location	language	follower_count
user_3o7w66o2	Berlin, Germany	ja	49944
user_u98jwp3f	Chicago, USA	zh	49936
user_usts5yuo	Shanghai, China	ja	49933
user_d4eat3v3	Tokyo, Japan	de	49914
user_siuvpkza	Chicago, USA	en	49905
user_ujllj1n7	Chicago, USA	es	49855

Returns: 293 rows, led by user_id=user_3o7w66o2, location=Berlin, Germany

MEDIUM questions

M1 - Engagement By Location

Question. Using both datasets, calculate the total engagement generated by users from each location. Return the location, number of posts, and total engagement. Rank locations from highest to lowest engagement.

Logic. The join is on user_id because engagement lives on the post while location lives on the user -- this is the one challenge that REQUIRES both datasets, and the query cannot run against either table alone. SUM uses v_posts_enriched.engagement, which is already NULL-safe (R1), so a location is never dropped for having a post with no like count. RANK() rather than ROW_NUMBER() is deliberate: tied locations should share a rank, and RANK is what a league table means.

location	n_posts	total_engagement	engagement_rank
Los Angeles, USA	391	1468866	1
Munich, Germany	388	1423109	2
Barcelona, Spain	376	1384741	3
Houston, USA	369	1331509	4
Dubai, UAE	360	1316970	5
Shanghai, China	374	1301061	6

Returns: 33 rows, led by location=Los Angeles, USA, n_posts=391

M1b - Engagement By Location

Question. (context for M1) Total engagement is a product of volume and quality. Separate the two.

Logic. Two RANK() windows are computed over the same rows: one on the total, one on the per-post average. If the two rankings disagree, then the "best location" answer is really "the location with the most posts", and saying more than that would be a volume artefact published as an insight. rank_shift quantifies the disagreement per row instead of leaving the reader to eyeball two columns.

location	n_posts	total_engagement	avg_engagement_per_post	rank_by_total	rank_by_avg_per_post	rank_shift
Los Angeles, USA	391	1468866	3756.7	1	1	0
Munich, Germany	388	1423109	3667.8	2	9	-7
Barcelona, Spain	376	1384741	3682.8	3	7	-4
Houston, USA	369	1331509	3608.4	4	19	-15
Dubai, UAE	360	1316970	3658.3	5	11	-6
Shanghai, China	374	1301061	3478.8	6	33	-27

Returns: 33 rows, led by location=Los Angeles, USA, n_posts=391

M2 - Do High Follower Users Get More Engagement?

Question. Divide users into two groups: high follower users ($\geq 25,000$ followers) and low follower users ($< 25,000$ followers). Compare their average engagement per post.

Logic. The grain is the POST, not the user. Averaging per-user averages would weight a user with 2 posts the same as a user with 17, which is the wrong unit for "average engagement per post" and would change the answer. The coefficient is computed from follower_count, a NOT NULL column, so no user can fall outside both cohorts. Standard deviation is reported next to each mean because it is the honest context for a difference of tens of engagements against a spread of thousands.

cohort	n_users	n_posts	avg_engagement_per_post	total_engagement	sd_engagement
LOW ($< 25k$ followers)	758	5176	3647.98	18881951	1718.9
HIGH ($\geq 25k$ followers)	742	5045	3598.82	18156054	1729.1

Returns: 2 rows, led by cohort=LOW ($< 25k$ followers), n_users=758

M2b - Do High Follower Users Get More Engagement?

Question. (context for M2) Say whether the gap in M2 is a finding.

Logic. The same two-sample z-statistic as E3b, here on the cohort contrast. This matters because the "obvious" business expectation (more followers > more engagement) is the one an analyst is most likely to confirm without a test. The z-statistic is what turns a directional difference into a claim, or refuses to.

low_follower_avg	high_follower_avg	difference	difference_pct	z_stat	verdict
3647.98	3598.82	-49.16	-1.35	-1.441	NOT significant at 5% -- follower count does not predict engagement here

Returns: 1 row: low_follower_avg=3,647.98, high_follower_avg=3,598.82

M3 - Most Active Users

Question. Find the top 10 users who have created the highest number of posts. Display their follower count, location, post count, and total engagement.

Logic. Post counts are small integers and therefore heavily tied (the busiest account wrote 17 posts). Without the extra sort keys the identity of the 10th user would depend on SQLite's internal row order. The ORDER BY is therefore total: post count, then total engagement, then user_id (R4). Total engagement is included as a displayed column and as the second sort key, so the ranking is "most active, and among equally active, most effective".

user_id	follower_count	location	n_posts	total_engagement
user_nfo3ih5u	40429	Barcelona, Spain	17	61730
user_n0ok02rt	2531	Dubai, UAE	16	49382
user_68enpikx	9364	Houston, USA	15	57816
user_xetn4exw	14902	Dubai, UAE	15	56209
user_8wk0j5ae	36943	Los Angeles, USA	15	51828
user_74ri5vbc	24934	New York, USA	15	49162

Returns: 10 rows, led by user_id=user_nfo3ih5u, follower_count=40429

M3b - Most Active Users

Question. (context for M3) Report the activity distribution so the top-10 cut can be judged.

Logic. A tie-aware census of post counts. If dozens of users share the count that sits on the 10th-place boundary, then "the top 10 most active users" is one arbitrary selection out of a much larger tied group, and the query says so with numbers instead of a caveat in prose.

busiest_user_posts	quietest_user_posts	tenth_place_posts	users_tied_at_the_cut	distinct_post_counts
17	1	14	12	17

Returns: 1 row: busiest_user_posts=17, quietest_user_posts=1

M4 - Platform Behaviour By High Follower Users

Question. Among users with at least 30,000 followers, determine which platform gives them the highest average engagement per post.

Logic. The cohort filter is applied to the USER before aggregation, so the comparison is between the same kind of account on each platform -- this is what separates M4 from E3, and it is the whole point of the challenge. 'Unspecified' is excluded because this query names a winner (R3); M4b shows it was not the winner anyway. RANK() exposes the whole ordering rather than a bare LIMIT 1, so the margin is visible.

platform	n_posts	avg_engagement_per_post	rank_within_cohort
Instagram	684	3692.1	1
Twitter	674	3583.7	2
YouTube	726	3569.9	3
Reddit	675	3561.3	4
Facebook	699	3532.1	5

Returns: 5 rows, led by platform=Instagram, n_posts=684

M4b - Platform Behaviour By High Follower Users

Question. (context for M4) The 30,000 threshold is arbitrary. Sweep it.

Logic. The challenge supplies 30,000 as a cut, but any cut is a choice. Five thresholds are swept in one pass with a VALUES list cross-joined into the aggregate, and ROW_NUMBER() picks the winner inside each threshold partition. If the same platform wins at every cut the finding is a property of the data; if the winner changes, the finding is a property of the threshold and must be reported as such.

follower_threshold	winning_platform	n_posts	avg_engagement_per_post
25000	Instagram	838	3658.3
30000	Instagram	684	3692.1
35000	Instagram	523	3689.7
40000	Instagram	366	3709.5
45000	Instagram	154	3706.1

Returns: 5 rows, led by follower_threshold=25000, winning_platform=Instagram

M5 - Detect Suspicious Engagement

Question. Find posts where the number of shares is greater than the number of likes and comments combined. Such posts may represent unusual sharing behaviour. Return the top 20.

Logic. The comparison "shares > likes + comments" is UNDEFINED for the 1,532 posts with no like count, so they are excluded by requiring likes to be present (R2). Treating a blank like count as 0 would manufacture 1,154 extra "anomalies" whose only defect is a missing field -- see M5b, which prices that mistake exactly. Ranking is by the size of the excess, the natural severity measure once both readings are ruled out; the ratio column is shown alongside because a 6-like post with 900 shares and a 490-like post with 2,000 shares are different phenomena.

post_id	user_id	platform	likes	shares	comments	excess_shares	share_to_reaction_ratio
603lc32fqk41	user_bq4d19qm	Instagram	145	1965	15	1805	12.28
8df7c2fopptde	user_vws4jlpj	Instagram	37	1915	118	1760	12.35
mtpfiec3whp4	user_mvu6hdsm	Unspecified	122	1880	92	1666	8.79
27w2t0eudwzl	user_cwh9v5dk	Twitter	6	1966	340	1620	5.68
ico83tj0cqa	user_rzirln2k	YouTube	61	1884	214	1609	6.85
agcvmdiq2joc	user_65x8ekq5	Instagram	91	1751	55	1605	11.99

Returns: 20 rows, led by post_id=603lc32fqk41, user_id=user_bq4d19qm

M5b - Detect Suspicious Engagement

Question. (context for M5) Show what the alternative reading would produce.

Logic. Three counts are placed side by side: the strict definition, the definition that reads a missing like count as zero, and the number of posts whose likes are missing AND whose shares exceed their comments. The third number is the gap between the first two, so the reader can see that every extra "anomaly" in the loose reading comes from missing data rather than from behaviour.

strict_definition	likes_read_as_zero	anomalies_created_by_blinks	arithmetic_gap	pct_of_loose_that_is_missing_data	reconciliation
1047	2201	1154	1154	52.43	every extra row in the loose reading is a missing like count

Returns: 1 row: strict_definition=1047, likes_read_as_zero=2201

HARD questions

H1 - Abnormally High Engagement

Question. Calculate the average total engagement per user. Identify users whose average engagement per post is more than twice the overall average engagement per post. Return the user ID, location, follower count, post count, and average engagement.

Logic. The baseline is the overall average engagement PER POST (every post weighted equally, which is what "overall average engagement per post" means on a post-level table), compared against each user's per-post average. This query returns ZERO rows, and that is the correct answer, not a bug: the arithmetic is laid out in H1b and swept in H1c. The column list is kept complete so the reader can see the shape of the answer that the data refuses to give.

user_id	location	follower_count	n_posts	avg_engagement
---------	----------	----------------	---------	----------------

The query executed successfully and returned 0 rows. The empty set is the arithmetic answer here, and the companion query underneath proves it rather than asserting it.

Returns: 0 rows - the empty set is the answer

H1b - Abnormally High Engagement

Question. (context for H1) An empty result must be shown to be arithmetic, not a broken query.

Logic. Every input to the decision is printed: the baseline, the doubled threshold, the highest per-user average that actually exists, and that maximum expressed as a multiple of the baseline. The verdict is computed, not asserted -- the database compares its own maximum against its own threshold. Reporting the near-miss multiple is what makes the emptiness informative rather than a dead end.

overall_avg_per_post	threshold_2x	users_examined	highest_user_avg	highest_avg_user	highest_avg_as_multiple_of_baseline	qualifying_users	verdict
3623.72	7247.43	1500	6545.5	user_2c5wzose	1.806	0	EMPTY BY ARITHMETIC -- no user reaches 2x, so H1 returning 0 rows is the answer

Returns: 1 row: overall_avg_per_post=3,623.72, threshold_2x=7,247.43

H1c - Abnormally High Engagement

Question. (context for H1) Find the multiple at which the question does have an answer, so the 2x cut can be reported against a real curve.

Logic. The same test is run at 1.25x, 1.50x, 1.75x and 2.00x in one compound statement. This converts "no users qualify" from a dead end into a threshold curve, which is the difference between refusing to answer and calibrating the question.

multiple	threshold	users_qualifying
1.25x	4529.65	145
1.50x	5435.57	8
1.75x	6341.5	1
2.00x	7247.43	0

Returns: 4 rows, led by multiple=1.25x, threshold=4,529.65

H2 - Rank Users Within Their Location

Question. For every location, rank users based on their total engagement. Return only the top 3 users from each location.

Logic. PARTITION BY location converts the global ranking into a per-location competition, which is the entire difficulty of the challenge. The tie-break chain (total engagement, then post count, then user_id) is not optional here: H2b shows that the 3rd and 4th place users in Tokyo are separated by 33 engagements out of ~43,000, so ties and near-ties decide the podium. ROW_NUMBER gives exactly 3 rows per location; RANK would be able to return more than 3 and silently break the challenge's "only the top 3" instruction.

location	rank_in_location	user_id	follower_count	n_posts	total_engagement
Barcelona, Spain	1	user_nfo3ih5u	40429	17	61730
Barcelona, Spain	2	user_24wzfb8b	38633	14	53039
Barcelona, Spain	3	user_5s9ifp0y	28412	10	51089
Beijing, China	1	user_cdzp4vm2	11187	11	48019
Beijing, China	2	user_z6qqqt5r	13029	8	38518
Beijing, China	3	user_gre0h06x	23303	9	36116

Returns: 99 rows, led by location=Barcelona, Spain, rank_in_location=1

H2b - Rank Users Within Their Location

Question. (context for H2) Show which locations have a contested 3rd place.

Logic. For each location the 3rd-place score is extracted with MAX(CASE WHEN rn = 3 ...) and the 4th-place score with MIN(CASE WHEN rn = 4...). The margin between them is the honesty check on the answer: a location whose margin rounds to 0.1% has a podium decided by luck, and reporting its top 3 without that number would overstate what the data knows.

location	third_place	fourth_place	absolute_margin	margin_pct
Tokyo, Japan	43092	43059	33	0.08
Dubai, UAE	49438	49382	56	0.11
Milan, Italy	42317	42255	62	0.15
Paris, France	39660	39550	110	0.28
New York, USA	41902	41684	218	0.52
Cairo, Egypt	38920	38702	218	0.56

Returns: 8 rows, led by location=Tokyo, Japan, third_place=43092

H3 - Platform Performance Vs Its Own Average

Question. For every platform, identify posts whose engagement is significantly higher than the average engagement of that platform. A post is considered exceptional if its engagement is at least 2x the average engagement of its platform.

Logic. The benchmark is each platform's OWN mean (a correlated per-platform aggregate joined back onto the posts), so a post on a low-engagement platform is judged against its peers rather than against the global mean -- that comparison is the point of the challenge. 'at least 2x' is inclusive, hence >=. 'Unspecified' is excluded because it has no platform identity to be exceptional within (R3); H3b reports the 5 posts that including it would have added.

post_id	user_id	platform	likes	shares	comments	platform_avg_engagement	post_engagement	multiple_of_platform_avg
wo7py9aljg3t	user_o8le7hqf	Reddit	4864	1981	948	3606.9	7793	2.161
ycjj5zzt7mvx	user_d9971ba6	Instagram	4983	1919	991	3673.2	7893	2.149
5kvuyvf38nqx	user_z0feut2e	YouTube	4923	1971	861	3621.5	7755	2.141
gmoeib832zbs	user_pe5yckyb	Facebook	4902	1880	982	3636.1	7764	2.135
a1kiwl618kzy	user_aaiari8o	Facebook	4811	1952	920	3636.1	7683	2.113
pvf13d8hj7jd	user_cs1uibwk	Instagram	4989	1840	909	3673.2	7738	2.107

Returns: 56 rows, led by post_id=wo7py9aljg3t, user_id=user_o8le7hqf

H3b - Platform Performance Vs Its Own Average

Question. (context for H3) Show each platform's bar and what the excluded bucket would have contributed.

Logic. The 2x bar is a derived number, not a given one, and it differs per platform (a platform's mean plus 100% of itself). Printing the bar for every platform makes the rule auditable and shows why the same post engagement can be exceptional on one platform and ordinary on another. 'Unspecified' is carried in the table but marked so its row is not read as a platform result (R3).

platform	n_posts	platform_avg_engagement	exceptional_threshold	exceptional_posts	pct_of_platform	eligibility
Twitter	1742	3561.5	7123.0	16	0.92	labelled platform
Reddit	1716	3606.9	7213.8	14	0.82	labelled platform
YouTube	1770	3621.5	7243.0	11	0.62	labelled platform
Facebook	1763	3636.1	7272.3	8	0.45	labelled platform
Instagram	1689	3673.2	7346.3	7	0.41	labelled platform
Unspecified	1541	3646.9	7293.8	5	0.32	DATA GAP -- excluded from the H3 answer

Returns: 6 rows, led by platform=Twitter, n_posts=1742

H4 - Follower To Engagement Anomaly

Question. Find users who have fewer than 5,000 followers but whose total post engagement places them among the top 10% of all users. These users may represent unusually high performing or suspicious accounts. This requires multiple levels of analysis.

Logic. Four levels, in order, and each one exists because the previous one is not enough: (1) collapse 10,221 posts to 1,500 users with SUM; (2) cut the user population into deciles with NTILE(10) on that total; (3) isolate the top decile; (4) re-join the followership dimension and apply the < 5,000 constraint. The joining key throughout is user_id, which is the only column linking the two datasets. NTILE assigns ranks before ties are considered, so users sharing a total can land in adjacent deciles -- H4b re-runs the same question with an explicit percentile threshold and shows the answer is unchanged.

user_id	location	follower_count	n_posts	total_engagement	avg_engagement_per_post	engagement_decile	top_decile_floor_engagement
user_uerv85na	Rome, Italy	1824	14	57044	4074.57	1	38765
user_hdas0iau	Rio de Janeiro, Brazil	1620	14	54678	3905.57	1	38765
user_fgjkkrie	Lyon, France	2211	12	53244	4437.0	1	38765
user_r7eg1rac	Houston, USA	2052	11	52606	4782.36	1	38765
user_n0ok02rt	Dubai, UAE	2531	16	49382	3086.38	1	38765
user_hvqqyzoi	Melbourne, Australia	3213	12	45668	3805.67	1	38765

Returns: 17 rows, led by user_id=user_uerv85na, location=Rome, Italy

H4b - Follower To Engagement Anomaly

Question. (context for H4) Two independent definitions of "top 10%" must agree before the finding is reported.

Logic. NTILE(10) forces exactly 10 equal buckets, which is convenient but splits tied totals arbitrarily at the boundary. The second definition is explicit: rank every user by total engagement and keep the top n*0.10 by ROW_NUMBER. If both definitions return the same users, the result is a property of the data rather than of the bucketing function. The decile floor is printed so the answer can be re-derived by hand.

top_decile_floor_engagement	via_ntile_10_buckets	via_explicit_percentile	robustness
38765	17	17	the two definitions AGREE -- the answer is not an artefact of NTILE

Returns: 1 row: top_decile_floor_engagement=38765, via_ntile_10_buckets=17

H5 - Identify Data Anomalies

Question. Identify potentially corrupted posts where one or more of the following conditions are true: (1) likes are negative; (2) platform is missing; (3) text content is missing; (4) text contains HTML entities/tags such as &, <div>, or
. Return the post ID and identify the type of anomaly.

Logic. This question CANNOT be asked of the cleaned table -- cleaning is exactly the step that removed these defects, so a query over `posts` would return nothing and prove nothing. It runs against `raw\_posts`, the verbatim intake staging table, and reads the defect definitions from v_raw_anomaly_scan, the same SQL predicate that populated the anomaly relation. The long-form relation means a post carrying three defects produces three rows, so no condition masks another; the counts are `>= 1` per condition rather than exclusive buckets. The concatenated label is built with CASE in a FIXED order (not GROUP_CONCAT, whose ordering is unspecified) so the output is byte-stable across re-runs.

post_id	n_anomalies	anomaly_types
0oqpg2rgnqxa	3	NEGATIVE_LIKES MISSING_PLATFORM MISSING_TEXT
26u0g2fea4nk	3	NEGATIVE_LIKES MISSING_PLATFORM MISSING_TEXT
5b0pz3j0kcxl	3	NEGATIVE_LIKES MISSING_PLATFORM MISSING_TEXT
6uafi1o2psiu	3	NEGATIVE_LIKES MISSING_PLATFORM HTML_ENTITY_OR_TAG
80rfm8lcs5rz	3	NEGATIVE_LIKES MISSING_PLATFORM MISSING_TEXT
95tln4xx75o	3	NEGATIVE_LIKES MISSING_PLATFORM MISSING_TEXT

Returns: 4418 rows, led by post_id=0oqpg2rgnqxa, n_anomalies=3

H5b - Identify Data Anomalies

Question. (context for H5) Reconcile the SQL scan against the Phase-1 corruption inventory.

Logic. The intake file contains 360 exact replays, so a defect can be counted two ways: rows in the file (what Phase 1's output/anomaly_table.csv reports) or distinct posts (what an anomaly table should contain). The difference column reconciles the two, and the arithmetic is checked against the file itself: 12,360 staged rows minus 360 replays must equal the distinct post count. A defect family whose intake count disagreed with Phase 1 would show up here as a mismatch rather than as a silent inconsistency between two artefacts.

anomaly_type	intake_rows	distinct_posts	replay_rows_removed
MISSING_PLATFORM	1846	1784	62
MISSING_TEXT	1770	1711	59
HTML_ENTITY_OR_TAG	1004	974	30
NEGATIVE_LIKES	525	509	16

Returns: 4 rows, led by anomaly_type=MISSING_PLATFORM, intake_rows=1846

H5c - Identify Data Anomalies

Question. (context for H5) Rank the worst-affected posts.

Logic. H5 shows every corrupted post; this isolates the multiply-corrupted ones, which are the rows most likely to break a naive pipeline and the ones worth eyeballing before any aggregate is trusted. Four defects in one row would mean the row is unusable for any purpose; three still leaves the post analytically alive, which is why the recovery kept them.

post_id	n_anomalies	anomaly_types
0oqpg2rgnqxa	3	NEGATIVE_LIKES MISSING_PLATFORM MISSING_TEXT
26u0g2fea4nk	3	NEGATIVE_LIKES MISSING_PLATFORM MISSING_TEXT
5b0pz3j0kcxl	3	NEGATIVE_LIKES MISSING_PLATFORM MISSING_TEXT
6uafi1o2psiu	3	NEGATIVE_LIKES MISSING_PLATFORM HTML_ENTITY_OR_TAG

post_id	n_anomalies	anomaly_types
80rfm8lcs5rz	3	NEGATIVE_LIKES MISSING_PLATFORM MISSING_TEXT
95tlcn4xx75o	3	NEGATIVE_LIKES MISSING_PLATFORM MISSING_TEXT

Returns: 20 rows, led by post_id=0oqpg2rgnqxa, n_anomalies=3

H6 - Most Suspicious High Impact Users

Question. Identify users who satisfy all three conditions: have fewer than 10,000 followers; their average post engagement is above the overall average; and at least one of their posts has more shares than likes. Rank these users by total engagement and display their location, follower count, number of posts, average engagement, and total engagement.

Logic. Three independent filters, each failing differently, so each is worth stating: (1) `follower\_count < 10000` is a property of the user; (2) the average is compared against the OVERALL average per post, the same baseline as H1, so the two hard questions in this set share one definition; (3) 'more shares than likes' is undefined without a like count, so those posts are excluded rather than counted as zero (R2) -- the same rule that M5b prices. Condition (3) is a set membership test, not an aggregate, which is why it is written as an IN subquery over DISTINCT users instead of another JOIN that could multiply rows.

location	follower_count	n_posts	avg_engagement_per_post	total_engagement	engagement_rank	user_id
Johannesburg, South Africa	8262	14	4458.14	62414	1	user_9mtets0p
Houston, USA	9364	15	3854.4	57816	2	user_68enpikx
Rio de Janeiro, Brazil	1620	14	3905.57	54678	3	user_hdas0ia u
Lyon, France	2211	12	4437.0	53244	4	user_fgjkkrie
Delhi, India	5153	11	4392.27	48315	5	user_zq2nib62
Vancouver, Canada	5971	10	4626.3	46263	6	user_acrn19py

Returns: 82 rows, led by location=Johannesburg, South Africa, follower_count=8262

H6b - Most Suspicious High Impact Users

Question. (context for H6) Show how many users each condition removes, so the answer can be seen to be selective rather than accidental.

Logic. A three-way AND that returns 82 users says nothing about whether the filters were meaningful or whether one of them did all the work. The funnel reports the population after each condition independently and then in combination, which is what makes the number interpretable -- and it is the same discipline H1 applies when it prints the baseline that made its own answer empty.

all_users	c1_fewer_than_10k_followers	c2_above_overall_avg	c3_has_over_shared_post	all_three_conditions
1500	287	748	1045	82

Returns: 1 row: all_users=1500, c1_fewer_than_10k_followers=287